



BOOTCAMP FRONTEND & REACT · SEMANA 3

ES6+

Clase 8 — ECMAScript moderno: template literals, desestructuración, spread y módulos

4 h 15 · L·M·V

Proyecto TechCart

STEERCLAVE

HOY NO HAY IDEAS NUEVAS: HAY UNA FORMA NUEVA DE ESCRIBIRLAS

Agenda

1. Repaso + los 2 difíciles
valor de una categoría · resumenCarrito

30'

2. Template literals
`\${ }` y el texto multilinea

26'

3. Destructuración
objetos, arrays y parámetros

37'

4. Spread y rest (...)
copiar en vez de mutar

42'

5. Más ES moderno
abreviadas · find · ?. · ??

38'

6. Módulos
export / import · un archivo, un trabajo

48'

7. Taller TechCart
agrupar por categoría · js/ui.js

20'

8. Cierre + 5 tareas
resumen, lecturas y anticipo

14'



Receso de 15 min entre el bloque 3 y el 4 · los ejercicios se cierran **por el chat**

Repaso exprés · Clase 7

Dos por el chat

`filter` me devuelve... ¿un array o un valor?

¿Qué es lo que todos olvidan al escribir un `reduce`?

LA DEUDA ES MÍA: NO LLEGUÉ A DEJARLAS, ASÍ QUE LAS RESOLVEMOS JUNTOS

Los 2 difíciles de la Clase 7

1) De los productos de la categoría "smartphones" **que tengan stock**, el **valor total** (precio × stock).

2) `resumenCarrito(items)` que devuelva un **objeto** con { cantidad, total }.

Los dos salen con lo de la clase pasada: `filter` + `reduce`.

```
// 1) filtro primero, acumulo después
const valorCategoria = (categoria) =>
  productos
    .filter(p => p.categoria === categoria &&
      p.stock > 0)
    .reduce((suma, p) => suma + p.precio *
      p.stock, 0)

valorCategoria("smartphones") // 8799.92
valorCategoria("tablets") // 0 ← el valor inicial
```

Cuatro piezas, cuatro símbolos

Lo mismo, escrito como se escribe hoy

``${ }``

armar textos
template literals

`{ }`

sacar datos
desestructuración

`...`

copiar sin romper
spread / rest

`import`

repartir el código
módulos

Las cuatro están en cada archivo de React. Hoy aprendemos su idioma.

COMILLAS INVERTIDAS ` ` · ADENTRO DEL \${ } CABE CUALQUIER EXPRESIÓN

Template literals

ANTES: CONCATENAR CON +

```
console.log("El " + p.nombre + " cuesta  
$" + p.precio)
```



AHORA: TEMPLATE LITERAL

```
console.log(`El ${p.nombre} cuesta  
${p.precio}`)
```

 **Java:** como `String.format`, pero el valor va **en su** lugar.

```
// decimales de dinero: .toFixed(2)  
`${(p.precio * 1.18).toFixed(2)} // $2359.99  
// y no $2359.9882
```

 **Caso real:** si la propiedad no existe, el texto no falla: escribe `undefined` o `NaN`. Así salen correos que dicen **"Gracias undefined, tu total es \$NaN"**. El error se va al cliente sin que nada se caiga.

CON `` EL TEXTO PUEDE OCUPAR VARIAS LÍNEAS · CON "" NO

La plantilla de una tarjeta

```
const fichaProducto = (p) => `  
  ${p.nombre}  
  categoría: ${p.categoría}  
  precio: $$${p.precio.toFixed(2)}  
  ${p.stock > 0 ? "En stock" : "Agotado"}  
`
```

Sigue siendo una arrow de **una sola expresión**: sin llaves ni return, aunque ocupe seis líneas.

🔗 **Java**: son los **text blocks** ("") de Java 15.

🐛 **Caso real**: antes había que escribir "
<article class=\"card\">" + con comillas
escapadas. Se cae una barra, el texto se corta y
la página queda vacía **sin ningún error**.

A LA IZQUIERDA DEL = VAN LAS PROPIEDADES QUE QUIERO

Desestructurar un objeto

lo largo

```
const nombre = p.nombre  
const precio = p.precio
```

en una línea

```
const { nombre, precio } = p
```

renombrar al sacar

```
const { nombre: titulo } = p
```

valor por defecto

```
const { descuento = 0 } = p
```

 **Caso real:** la API manda un producto sin descuento, la cuenta da NaN y la tienda muestra "Precio: \$NaN". El `= 0` es la red de seguridad más corta que existe.

Arrays, y el caso que **importa**

arrays – por posición

```
const [primero, segundo] = productos
const [, , tercero] = productos // salta 2
```

en el parámetro de la función

```
const ficha = ({ nombre, categoria, precio }) =>
  `${nombre} (${categoria}) - ${precio}`

// y dentro de un map:
productos.map(({ nombre }) => nombre)
```

Se lee como un **contrato**: "esta función recibe un objeto que tenga nombre, categoría y precio".

 **Caso real:** Cannot destructure property of undefined es la causa nº 1 de **pantalla en blanco**: el componente se dibuja antes de que lleguen los datos de la API.

 EJERCICIO · MÁX. 6 MIN · ME PEGAN LA PRIMERA LÍNEA AL CHAT

Moderno de punta a punta

Armen resúmenes, donde cada elemento sea:

```
"MacBook Pro 14 (laptops): $1999.99"
```

Con **map**, **desestructurando** en el parámetro (nada de p. adentro) y con **template literal**.
Precio con **toFixed(2)**.

```
// solución
const resúmenes = productos.map(
  ({ nombre, categoria, precio }) =>
    `${nombre} (${categoria}):
    ${precio.toFixed(2)}`
)
```

... DESPARRAMA EL CONTENIDO DE UN ARRAY DENTRO DE OTRO

Spread: copiar, agregar, combinar

1999

1099

499

[...productos, nuevo]

1999

1099

499

79

productos.push(x)

Muta: el array original ya no es el que era. Quien lo comparta se enteró sin pedirlo.

[...productos, x]

Crea **uno nuevo** y deja el original en paz. Puedo comparar, deshacer o botarlo.

🔗 **Java:** `List.copyOf` y `Stream.concat` — la misma discusión de listas mutables vs inmutables.

COPIO TODO Y PISO UNA PROPIEDAD · EL ÚLTIMO QUE SE ESCRIBE, GANA

Spread en objetos

```
const rebajado = { ...productos[0], precio: 1799.99 }
```

```
productos[0].precio // 1999.99 ← intacto  
rebajado.precio // 1799.99  
rebajado.nombre // se copió solo
```

y los mutadores que hay que vigilar

```
productos.sort(...) // ❌ muta  
[...productos].sort(...) // ✅ copia y ordena
```

💡 **Caso real (React):** alguien hace `carrito.push(p)`, el dato **sí** entra pero **la pantalla no cambia**. No hubo objeto nuevo, así que React nunca se enteró. La solución es `[...carrito, p]`.

💡 **Y el del sort:** el usuario ordena por precio, después quita el orden... y el catálogo no vuelve, porque el array original quedó reordenado para siempre.

⚠️ El spread copia **un nivel**. Objetos anidados se comparten (copia superficial).

LOS MISMOS TRES PUNTOS HACEN LO CONTRARIO SEGÚN DÓNDE ESTÉN

Rest: juntar lo que llegue

SPREAD — DESPARRAMA

```
const precios =  
productos.map(p =>  
  p.precio)  
Math.max(...precios)  
// 1999.99
```

REST — JUNTA

```
const sumar =  
(...numeros) =>  
  numeros.reduce((s,  
    n) => s + n, 0)  
sumar(10, 20, 30) //  
60
```

 **Java:** esto son los **varargs**, `int... numeros`.
Misma idea y misma sintaxis de puntos.

 **Caso real:** rest al desestructurar sirve para **quitar un campo** antes de mandar datos afuera: `const { password, ...datosSeguros } = usuario`.

TRES DETALLES CORTOS QUE APARECEN EN TODO EL CÓDIGO MODERNO

Abreviadas, objeto y defaults

`{ cantidad, total }`

Propiedades **abreviadas**: si la propiedad se llama igual que la variable, se escribe una vez.

```
// antes
{ cantidad: cantidad }
// ahora
{ cantidad }
```

`p => ({ ... })`

Arrow que devuelve un **objeto**: hay que envolverlo en **paréntesis**.

```
// sin ( ) devuelve
undefined:
p => { etiqueta: p.nombre } ❌
p => ({ etiqueta:
p.nombre }) ✅
```

`(precio, pct = 10)`

Parámetros **por defecto**.  Java lo resuelve con **sobrecarga**; acá es `un = 10`.

```
conDescuento(1000) // 900
conDescuento(1000, 25) // 750
```

 JAVA: FINDFIRST · ANYMATCH · ALLMATCH

find · some · every

find

devuelve **el primer** elemento que cumple... o undefined

some

¿hay **al menos uno**? → true / false

every

¿cumplen **todos**? → true / false

```
productos.some(p => p.stock === 0) // true
productos.every(p => p.stock > 0) // false
```

 **Caso real:** nunca busquen con find por **nombre**. En un catálogo real hay dos "Cargador USB-C" y devuelve el primero: el usuario quita un producto del carrito y desaparece **otro**. Se busca por id.

 **Y para decidir en la interfaz** usen some, no filter: some para en cuanto encuentra el primero; filter recorre todo y arma un array que van a botar.

🔗 EL NULLPOINTEREXCEPTION DE JAVASCRIPT, Y SUS DOS REDES

?. y ??

```
const x = productos.find(p => p.nombre === "Switch")
x.precio // ✗ TypeError · el script SE DETIENE
x?.precio // undefined, sin error
x?.precio ?? 0 // 0
```

stock || "sin dato"

Con stock: 0 da "sin dato". El 0 es un dato válido. ✗

stock ?? "sin dato"

Da 0. Solo salta con null y undefined. ✓

🦋 **Caso real:** traemos 38 productos de una API. Basta con que **uno** no traiga imagen para que `p.imagen.url` reviente, y no se cae una tarjeta: **se cae el catálogo completo**, porque el script se detiene.

Con `p.imagen?.url ?? porDefecto`, ese producto sale con imagen genérica y los otros 37 salen bien.

LA REGLA QUE YA CONOCEN DE JAVA: UNA CLASE, UN ARCHIVO

Un archivo, **una responsabilidad**

datos.js

el catálogo. Única fuente de verdad de los productos.

carrito.js

el dinero: IGV, resumen, descuentos. Los productos le llegan por parámetro.

ui.js

cómo se ve un producto. No calcula nada.

main.js

orquestra: pide, usa y muestra. No sabe hacer nada solo.



Caso real: el archivo revuelto no se queda en 100 líneas, llega a 2000. Y pasa algo peor que la incomodidad: **nadie lo quiere tocar**, porque cambiar el IGV significa entrar donde también viven la búsqueda, el render y la sesión. Ese archivo se vuelve intocable y el equipo empieza a copiar funciones a otros lados. Partirlo hoy cuesta diez minutos; con 2000 líneas, no se hace nunca.

EXPORT = PUBLIC · LO QUE NO SE EXPORTA SE QUEDA ADENTRO DEL ARCHIVO

export / import

```
// js/datos.js
export const productos = [ ... ]

// js/main.js – con llaves, por nombre
import { productos } from "./datos.js"
import { resumenCarrito } from "./carrito.js"

// export default: UNO por archivo, sin llaves
export default formatearPrecio
import formatearPrecio from "./formato.js"
```

```
<!-- index.html: el interruptor -->
<script type="module" src="js/main.js"></script>
```

los 3 errores que van a ver hoy

Cannot use import statement...

falta type="module"

404 / Failed to resolve

falta la extensión .js (es obligatoria)

does not provide an export named...

la función existe pero falta el export

💣 **El traicionero:** `./Datos.js` con mayúscula **funciona en Windows** y da 404 en Vercel (Linux). Es el clásico "en mi máquina funcionaba". Nombres de archivo en minúscula, siempre.

 EJERCICIO · MÁX. 6 MIN · Y DESPUÉS BORREN EL EXPORT A PROPÓSITO

Su propio módulo

En `carrito.js`, agreguen y **exporten** `masCaroDe(items)`: el producto de mayor precio.

Es un `reduce` donde el acumulador **no es un número**, es un producto: "el más caro hasta ahora".

Impórtenla en `main.js` e impriman nombre y precio.

```
// js/carrito.js
export const masCaroDe = (items) =>
  items.reduce(
    (mayor, p) => (p.precio > mayor.precio ? p :
    mayor),
    items[0]
  )

// js/main.js — UNA línea de import, no tres
import { resumenCarrito, conDescuento, masCaroDe
}
from "./carrito.js"
```

Agrupar el catálogo por categoría

```
// js/datos.js - ¿en qué archivo va? No es dinero.
export const contarPorCategoria = (items) =>
  items.reduce((acc, p) => ({
    ...acc,
    [p.categoria]: (acc[p.categoria] ?? 0) + 1
  }), {})

// { laptops: 1, smartphones: 1, tablets: 1, audio:
  2 }
```

```
// ¿y la LISTA de nombres por categoría?
[p.categoria]: [...(acc[p.categoria] ?? []),
p.nombre]
```

Valor inicial `{ }`: lo que armo es un objeto.

`...acc`: copio lo que ya tenía (sin mutar).

`?? 0`: la primera vez esa categoría no existe.

`[p.categoria]`: los corchetes hacen que JS evalúe el nombre de la propiedad.



Caso real: agrupar es de lo que más se pide en front-end y casi nunca lo hace el backend: pedidos por mes para un gráfico, productos por marca para los filtros, tareas por estado para un tablero.

HOY NO APRENDIMOS IDEAS NUEVAS: APRENDIMOS A ESCRIBIRLAS COMO SE ESCRIBEN HOY

Resumen

``${ }``

template literals · y multilínea

`{ nombre, precio } = p`desestructurar · sobre todo en el
parámetro`[...arr, x]`

spread: copiar en vez de mutar

`(...numeros)`

rest = los varargs de Java

`find · some · ?. · ??`

buscar, preguntar y no reventar

`export / import`

un archivo, una responsabilidad

LAS DOS DIFÍCILES SON LAS DOS MITADES DE LA PRÓXIMA CLASE · LA QUE NADIE LOGRE LA VEMOS JUNTOS

5 tareas

FÁCIL

Una línea por producto: **"AirPods Max · audio · \$549.99"** — map + desestructurar + `formatearPrecio`.

INTERMEDIO

aplicarDescuento(producto, pct = 10): devuelve una **copia** rebajada. Demuestren que el original no cambió.

INTERMEDIO

buscarPorNombre(items, nombre) con `find`: cuando no exista, **"No lo tenemos · \$0.00"** con `?.` y `??`.

DIFÍCIL

En `ui.js`: **listaCatalogo(items)** (un solo texto con todas las fichas) y **resumenStock(items)** → "5 productos, 1 agotado".

DIFÍCIL

agregarAlCarrito(carrito, producto) con `spread`, sin `push`, y **resumenCarrito** con `{ cantidad, subtotal, igv, total }`.

Próxima clase

La tienda cobra vida

Se **ve** desde la Clase 6, **piensa** desde la 7, y desde hoy piensa en el idioma correcto y con el código repartido en archivos. Pero todo sigue en la **consola**. La próxima clase abrimos la puerta entre JavaScript y la página: el **DOM**. Borramos las tarjetas escritas a mano y el catálogo se **pinta solo** desde el array, los **botones responden** al clic y el carrito **sobrevive al refresco**.

Pista: en la consola, `document.querySelector("h1").textContent = "Mi tienda"`